



Proyecto Final Capstone

Consumo y visualización de datos desde una API en contenedor

Fecha: 12/7/2026

Estudiante: Tomas Urbicain

D.N.I.: 47.958.639

Carrera: Tecnicatura Universitaria en Programación

Materia: Arquitectura y Sistemas Operativos

Índice

Introducción	3
Conceptos Teóricos	3
Desarrollo	4
Diagrama de Node-RED.....	5
Diagrama en bloques	6

Introducción

El presente trabajo integra el uso de contenedores **Docker** y flujos en **Node-RED** para el consumo y procesamiento de datos en tiempo real mediante una API externa.

Conceptos Teóricos

Docker

Docker es una herramienta fundamental en la computación moderna que permite la virtualización y creación de contenedores.

En nuestro proyecto final, se utiliza para empaquetar toda la solución (Node-RED, configuraciones y dependencias) dentro de entornos aislados y portátiles llamados contenedores

La función principal de la herramienta Docker es asegurar que el sistema funcione de la misma manera en cualquier computadora mediante un archivo *docker-compose.yml* en el que se le indica todos los pasos e imágenes que precisa la solución completa. En cada contenedor hay un proceso o servicio (para nuestro caso es el Node-RED)

API REST

Una API REST es una interfaz estándar que permite la comunicación entre sistemas a través de la web, funcionando como un mecanismo de transporte de datos.

Se consulta generalmente mediante protocolos estándar como HTTPS, realizando peticiones (como un "GET") a un endpoint o URL específica.

En este trabajo, la consulta se realiza mediante el nodo ***http request*** de Node-RED, el cual recibe la información en formato JSON, permitiendo que el sistema obtenga datos de dispositivos remotos de forma estructurada.

Sistema IoT

Un sistema IoT (Internet de las Cosas) se caracteriza por conectar objetos físicos a una red para que puedan recolectar información.

Los componentes clave representados en un sistema IoT son el origen de los datos (dispositivo IoT), el transporte (red/API) y la visualización en pantalla.

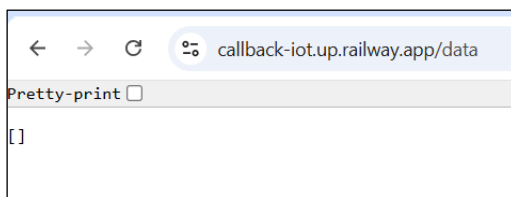
Este tipo de sistemas genera datos reales relacionados con variables físicas. En general se trata de registros de sensores que incluyen marcas de tiempo (fecha/hora) y mediciones como temperatura, humedad y presión.

Node-RED

Node-RED actúa como la plataforma de procesamiento intermedio de datos. Su papel principal es actuar como un motor de flujo que permite disparar un evento cada cierto tiempo, consultar via API REST al sistema IoT, procesar los datos y mostrarlos en pantalla.

Desarrollo

Lo primero que debemos aclarar es que el endpoint sugerido en el anuncio (<https://callback-iot.up.railway.app/data>) no devolvía datos. La respuesta era una estructura json vacía.



Por lo que se tomó la decisión de buscar algún otro servidor API REST público y gratuito, pero además que cambie sus valores de forma rápida para poder desarrollar el proyecto final de forma completa y con una buena visualización de datos en los widgets.

Por este motivo, se eligió el sitio Binance que publica los valores de las criptomonedas públicamente y con una alta frecuencia de refresco. Se tomaron 2 criptomonedas para realizar el trabajo final que son Bitcoins y Ethereum.

Con la siguiente consulta [https](https://api.binance.com/api/v3/ticker/price?symbols=[\)

[https://api.binance.com/api/v3/ticker/price?symbols=\[\"BTCUSDT\",\"ETHUSDT\"\]](https://api.binance.com/api/v3/ticker/price?symbols=[\)

obtenemos los datos con la siguiente estructura json.

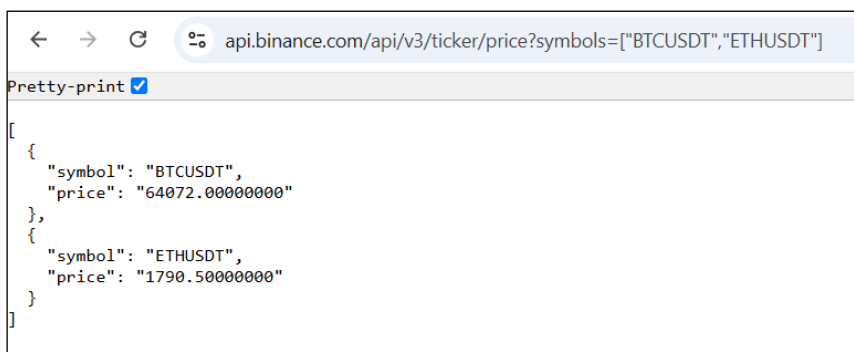
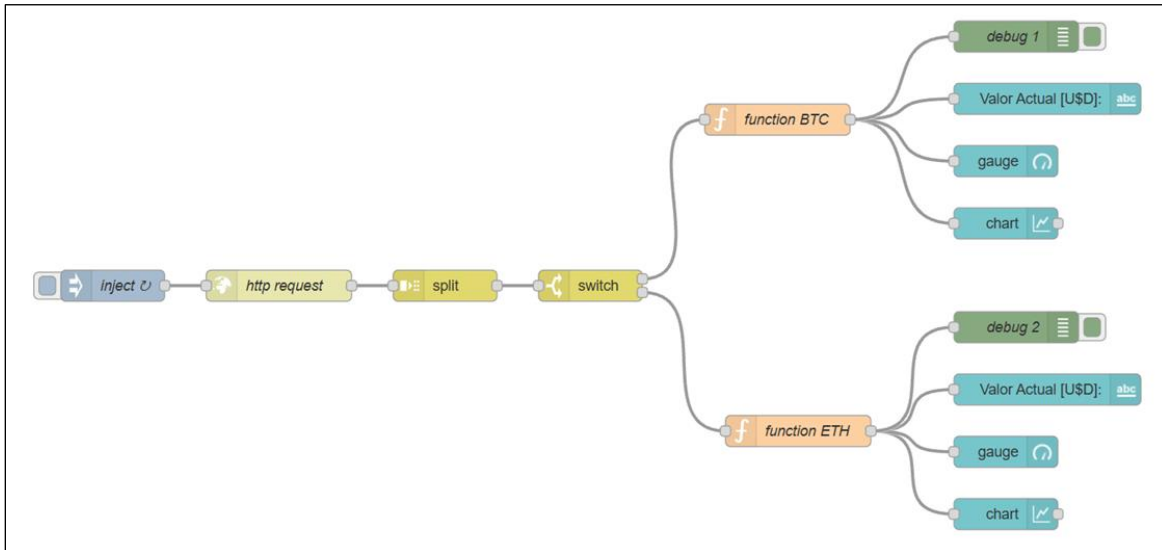


Diagrama de Node-RED

En el siguiente diagrama se ve el flujo de nuestro sistema



A continuación, una explicación de cada nodo:

- **Nodo Inject:** Es el que dispara un evento de forma automática cada **3 segundos**, garantizando la actualización de los datos.
- **Nodo HTTP Request:** Se encarga de realizar la petición **GET** a la URL de Binance mediante el protocolo **HTTPS**. Su función es obtener el paquete de datos en formato **JSON**.
- **Nodo Split:** Actúa como un traductor que convierte la cadena de texto json recibida de la API en un objeto estructurado, permitiendo que los nodos siguientes puedan leer y manipular los valores de precios.
- **Nodo Switch:** Funciona como un separador. Evalúa la propiedad del mensaje según el símbolo de la moneda y encamina los datos hacia salidas diferentes, permitiendo separar el precio de **Bitcoin** del de **Ethereum**.
- **Nodos Gauge, Chart y Text:** Representan la capa de visualización final del sistema. El **Gauge** muestra el valor actual de forma analógica, mientras que el **Chart** genera un gráfico de líneas para visualizar la tendencia. Existe un tercer widget **Text** que muestra el valor numérico. Para definirle los límites mínimo y máximo a Gauge y al Chart, se tomó el 1%

menos y más respectivamente del valor inicial al momento de hacer el deploy para fijar los límites, de esta forma ambos widgets se ven correctamente.

- **Nodos Debug:** Este nodo permite ver los valores en la ventana de Debug para estudiar posibles problemas.

En la siguiente imagen vemos como quedaría la visualización final del panel de comando

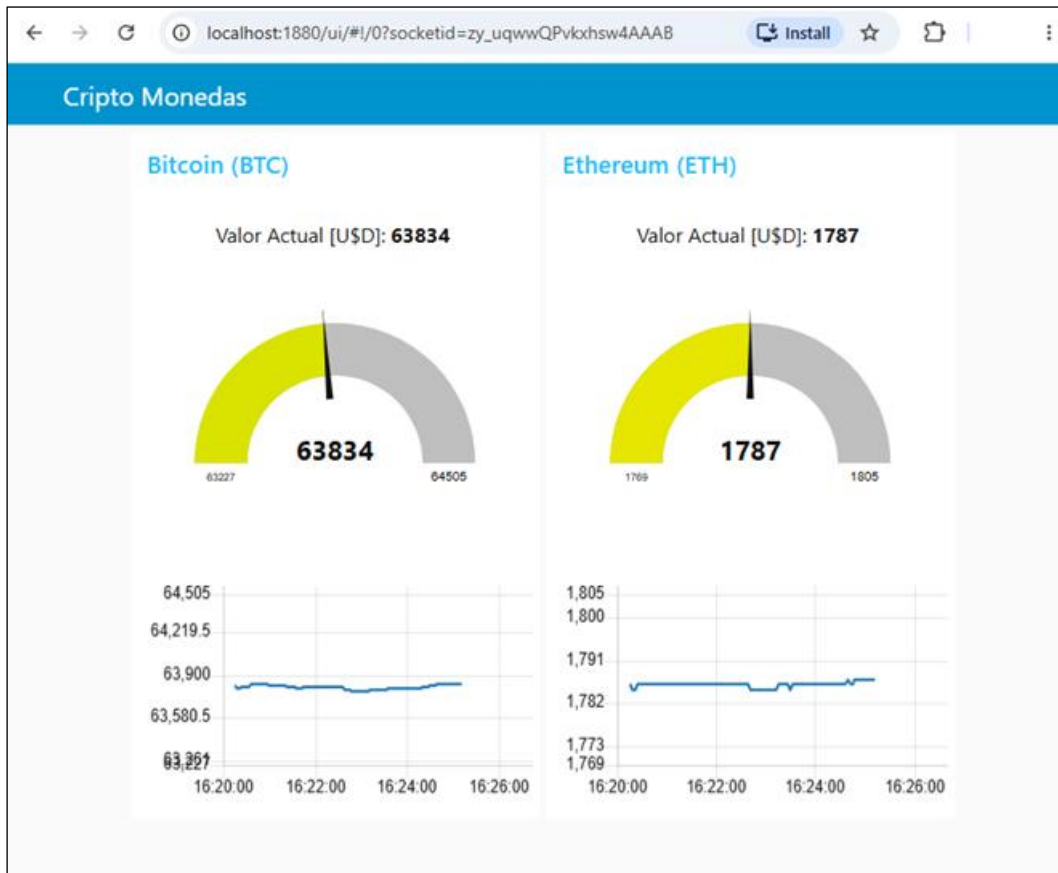
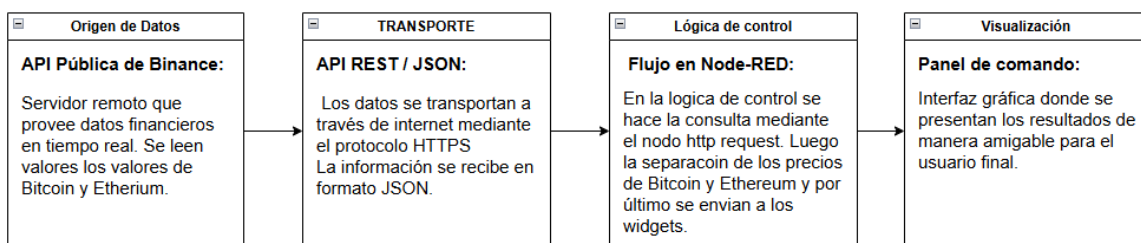


Diagrama en bloques

A continuación, el diagrama en bloque de nuestro sistema



Reflexión sobre dificultades y aprendizajes

Como se comentó al comienzo del documento, tuve dificultades para poder obtener los datos de IoT propuesto por el enunciado del trabajo, pero gracias las herramientas que se cuenta hoy día en internet fue posible buscar otra API para poder desarrollar el trabajo. Tras investigar diversas alternativas (como los límites de consulta, frecuencias altas, etc) se optó por la API de Binance.

Con respecto a los aprendizajes, el trabajo permitió aprender sobre la contenerización con Docker, entendiendo que con esta tecnología la solución sea portable y funcione de manera idéntica en cualquier computadora mediante el archivo *docker-compose.yml*.

Asimismo, el uso de Node-RED fue fundamental para entender la lógica de procesamiento intermedio. Aprendí que mediante nodos como SPLIT y SWITCH es posible separar los flujos de datos para presentarlos de forma organizada en un tablero de control. El Node-RED permite de forma visual programar tareas de forma sencilla.